Universidade Estadual de Santa Cruz – UESC

Relatórios de Implementações de Métodos da Disciplina Análise Numérica

Relatório de implementações
realizadas por Lucas Vieira de Almeida

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2025.2

Professor Gesil Sampaio Amarante II

Ilhéus – BA
2025

Índice

1. Linguagem(ns) escolhida(s) e justificativas.....7

2. Método da bissecção.....8

2.1. Estratégia de implementação.....8

2.2. Estrutura dos arquivos de entrada/saída.....9

2.3. Dificuldades enfrentadas.....9

3. Método da posição falsa.....10

3.1. Estratégia de implementação.....10

3.2. Estrutura dos arquivos de entrada/saída.....10

3.3. Dificuldades enfrentadas.....10

4. Método de Newton-Raphson..... 12

4.1. Estratégia de implementação.....12

4.2. Estrutura dos arquivos de entrada/saída.....12

4.3 Dificuldades enfrentadas.....12

5. Método da secante..... 14

5.1. Estratégia de implementação.....14

5.2. Estrutura dos arquivos de entrada/saída.....14

5.3. Dificuldades enfrentadas.....14

6. Problemas Estabelecidos para raízes de funções.....16

6.1. Problema 1 (Exercício 3.3).....16

6.2. Problema 2 (Exercício 3.6).....22

6.3. Problema 3 (Exercício 3.8).....26

7. Comparativo…….....32

8. Método da Eliminação de Gauss.....34

8.1. Estratégia de implementação.....34

8.2. Estrutura dos arquivos de entrada/saída.....34

8.3. Dificuldades enfrentadas.....35

9. Método da fatoração LU.....36

9.1. Estratégia de implementação.....36

9.2. Estrutura dos arquivos de entrada/saída.....36

9.3. Dificuldades enfrentadas.....37

10. Problemas Estabelecidos para solução de sistemas lineares..... 38

10.1. Problema 1 (Exercício 4.1).....38

10.2. Problema 2 (Exercício 4.3).....41

10.3. Problema 3 (Exercício 4.6).....43

11. Comparativo…….....45

12. Método de Jacobi……..... 46

12.1. Estratégia de implementação.....46

12.2. Estrutura dos arquivos de entrada/saída.....46

12.3. Dificuldades enfrentadas.....47

13. Método de Gauss-Seidel…….....48

13.1. Estratégia de implementação.....48

13.2. Estrutura dos arquivos de entrada/saída.....48

13.3. Dificuldades enfrentadas.....49

14. Problemas Estabelecidos para solução de sistemas lineares.....50

14.1. Problema 1 (Exercício 5.1).....50

14.2. Problema 2 (Exercício 5.2).....52

14.3. Problema 3 (Exercício 5.5).....55

15. Comparativo…….....57

16. Número de condição das matrizes.....58

16.1. Estratégia de implementação.....58

16.2. Estrutura dos arquivos de entrada/saída.....58

16.3. Dificuldades enfrentadas.....59

17. Problemas Estabelecidos para número de condição das matrizes.....60

17.1. Problema 1 (Exercício 4.1).....60

17.2. Problema 2 (Exercício 4.3).....61

17.3. Problema 3 (Exercício 4.6).....62

17.4. Problema 4 (Exercício 5.1).....63

17.5. Problema 5 (Exercício 5.2).....64

17.6. Problema 6 (Exercício 5.5).....65

18. Considerações finais.....66

Lista de Figuras

1. Figura 1 – Entrada 1 para o problema 1 (Exercício 3.3).....16

2. Figura 2 – Entrada 2 para o problema 1 (Exercício 3.3).....17

3. Figura 3 – Saída 1 Bissecção para o problema 1 (Exercício 3.3).....17

4. Figura 4 – Saída 1 Posição Falsa para o problema 1 (Exercício 3.3).....18

5. Figura 5 – Saída 1 Newton-Raphson para o problema 1 (Exercício 3.3).....18

6. Figura 6 – Saída 1 Secante para o problema 1 (Exercício 3.3).....19

7. Figura 7 – Saída 2 Bissecção para o problema 1 (Exercício 3.3).....19

8. Figura 8 – Saída 2 Posição Falsa para o problema 1 (Exercício 3.3).....20

9. Figura 9 – Saída 2 Newton-Raphson para o problema 1 (Exercício 3.3).....20

10. Figura 10 – Saída 2 Secante para o problema 1 (Exercício 3.3).....21

11. Figura 11 – Entrada para o problema 2 (Exercício 3.6).....23

12. Figura 12 – Saída Método Bissecção para problema 2 (Exercício 3.6).....23

13. Figura 13 – Saída Posição Falsa para problema 2 (Exercício 3.6).....24

14. Figura 14 – Saída Newton-Raphson para problema 2 (Exercício 3.6).....24

15. Figura 15 – Saída Método Secante para problema 2 (Exercício 3.6).....25

16. Figura 16 – Entrada 1 para o Problema 3 (3.8).....26

17. Figura 17 – Entrada 2 para o Problema 3 (3.8).....27

18. Figura 18 – Saída 1 Bissecção para o Problema 3 (3.8).....27

19. Figura 19 – Saída 2 Bissecção para o Problema 3 (3.8).....28

20. Figura 20 – Saída 1 Posição Falsa para o Problema 3 (3.8).....28

21. Figura 21 – Saída 2 Posição Falsa para o Problema 3 (3.8).....29

22. Figura 22 – Saída 1 Newton-Raphson para o Problema 3 (3.8).....29

23. Figura 23 – Saída 2 Newton-Raphson para o Problema 3 (3.8).....30

24. Figura 24 – Saída 1 Secante para o Problema 3 (3.8).....30

25. Figura 25 – Saída 2 Secante para o Problema 3 (3.8).....31

26. Figura 26 – Circuito do problema 1 (Exercício 4.1).....38

27. Figura 27 – Sistema Linear para o problema 1 (Exercício 4.1).....38

28. Figura 28 – Entrada para o problema 1 (Exercício 4.1).....39

29. Figura 29 – Saída Gauss para o problema 1 (Exercício 4.1).....39

30. Figura 30 – Saída LU problema 1 (Exercício 4.1).....40

31. Figura 31 – Labirinto do problema 2 (Exercício 4.3).....41

32. Figura 32 – Equações para Pi do problema 2 (Exercício 4.3).....41

33. Figura 33 – Entrada para o problema 2 (Exercício 4.3).....42

34. Figura 34 – Saída Gauss para o problema 2 (Exercício 4.3).....42

35. Figura 35 – Saída LU problema 2 (Exercício 4.3).....42

36. Figura 36 – Malha para resolução do problema 3 (Exercício 4.6).....43

37. Figura 37 – Sistema linear do problema 3 (Exercício 4.6).....43

38. Figura 38 – Entrada para o problema 3 (Exercício 4.6).....43

39. Figura 39 – Saída Gauss para o problema 3 (Exercício 4.6).....44

40. Figura 40 – Saída LU problema 3 (Exercício 4.6).....44

41. Figura 41 – Sistema linear do problema 1 (Exercício 5.1).....50

42.Figura 42 – Entrada para o problema 1 (Exercício 5.1).....50

43.Figura 43 – Saída Gauss-Seidel do problema 1 (Exercício 5.1).....51

44.Figura 44 – Saída Jacobi do problema 1 (Exercício 5.1).....51

45.Figura 45 – Malha para resolução do problema 2 (Exercício 5.2).....52

46.Figura 46 – Sistema linear para o problema 2 (Exercício 5.2).....52

47.Figura 47 – Entrada para o problema 2 (Exercício 5.2).....53

48.Figura 48 – Saída Gauss-Seidel do problema 2 (Exercício 5.2).....53

49.Figura 49 – Saída Jacobi do problema 2 (Exercício 5.2).....54

50.Figura 50 – Imagem para a resolução do problema 3 (Exercício 5.5).....55

51.Figura 51 – Sistemas de equação para o problema 3 (Exercício 5.5).....55

52.Figura 52 – Entrada para o problema 3 (Exercício 5.5).....56

53.Figura 53 – Saída Gauss-Seidel do problema 3 (Exercício 5.5).....56

54.Figura 54 – Saída Jacobi do problema 3 (Exercício 5.5).....56

55.Figura 55 – Entrada para o código de condição do problema 4.1.....60

56.Figura 56 – Saída da condição da matriz do problema 4.1…….....60

57.Figura 57 – Entrada para o código de condição do problema 4.3.....61

58.Figura 58 – Saída da condição da matriz do problema 4.3.....61

59.Figura 59 – Entrada para o código de condição do problema 4.6.....62

60.Figura 60 – Saída da condição da matriz do problema 4.6.....62

61.Figura 61 – Entrada para o código de condição do problema 5.1.....63

62.Figura 62 – Saída da condição da matriz do problema 5.1.....63

63.Figura 63 – Entrada para o código de condição do problema 5.2.....64

64.Figura 64 – Saída da condição da matriz do problema 5.2.....64

65.Figura 65 – Entrada para o código de condição do problema 5.5.....65

66.Figura 66 – Saída da condição da matriz do problema 5.5.....65

1. Linguagem(ns) Escolhida(s) e Justificativas

A linguagem Python foi escolhida para a implementação dos métodos numéricos porque apresenta características que facilitam tanto o desenvolvimento quanto a compreensão do código. Sua sintaxe simples e próxima da matemática permite que os algoritmos sejam escritos de forma clara, sem excesso de comandos técnicos, tornando o foco principal a lógica dos métodos e não detalhes de programação.

Outro ponto importante é o uso de bibliotecas como o NumPy, que oferece recursos otimizados para manipulação de vetores e matrizes, operações fundamentais em métodos de álgebra linear e cálculo numérico. Isso garante mais rapidez e precisão nos resultados, além de reduzir o esforço necessário para implementar operações matemáticas complexas.

Nos métodos de raízes, a presença do Sympy também foi essencial, pois possibilita derivar e avaliar funções automaticamente, evitando erros de cálculo manual e tornando os algoritmos mais versáteis e práticos.

Além disso, Python possui ferramentas simples para medir tempo de execução, o que ajuda a comparar o desempenho dos diferentes métodos de forma direta.

Por fim, Python é uma linguagem muito utilizada no meio acadêmico e científico, com ampla documentação e comunidade ativa, o que facilita o aprendizado e a resolução de dúvidas. Dessa forma, sua escolha garante não apenas eficiência na implementação, mas também maior acessibilidade e compreensão dos conceitos estudados.

2. Método Bissecção

2.1 Estratégia de Implementação

A implementação do método da bissecção foi estruturada dentro de uma classe Raízes que encapsula múltiplos métodos numéricos, permitindo reutilização de código e consistência na interface. Esta abordagem orientada a objetos facilita a manutenção e expansão do sistema. Inicialmente, é verificado se os valores da função nos extremos do intervalo [a, b] possuem sinais opostos. Essa condição é fundamental, pois garante a existência de pelo menos uma raiz dentro do intervalo (Teorema do Valor Intermediário). No código, essa etapa é implícita: o método não prossegue a não ser que, ao longo das iterações, seja identificado em qual subintervalo a raiz se encontra. Caso $f(a) \cdot f(b) > 0$, o método não terá como convergir.

Nos casos em que a raiz está presente, o algoritmo prossegue calculando sucessivos pontos médios $c = (a+b) / 2$. O critério de parada é verificado a cada iteração de duas formas:

- se $|f(c)| < tol$, indicando que o valor do ponto médio está suficientemente próximo da raiz; ou

- se $(b-a) / 2 < tol$, garantindo que o intervalo de busca já está suficientemente reduzido.

Assim, assegura-se que a tolerância estipulada pelo usuário seja respeitada, caso o usuário não especifique no arquivo de entrada, o código funciona adequadamente utilizando uma tolerância padrão 1e-8 que balanceia precisão e desempenho, já que Python tem precisão limitada de ponto flutuante.

O cálculo do erro adotado é baseado no valor da função ou no comprimento do intervalo, estratégia escolhida para garantir convergência mesmo em funções menos inclinadas próximas à raiz. O resultado final (raiz aproximada, erro associado, número de iterações e tempo de execução) é registrado no arquivo de saída, permitindo analisar a eficiência do processo.

2.2 Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada segue um formato simples e direto. Na primeira linha, é fornecida a expressão da função (em notação compatível com o **Sympy**), e nas linhas seguintes os valores numéricos para o intervalo [a,b]. Opcionalmente, uma terceira linha pode conter a tolerância desejada. Como explicado na estratégia, caso não tenha é utilizado uma tolerância padrão do código. Esse formato foi escolhido pela facilidade de leitura e edição, além de permitir que funções arbitrárias sejam interpretadas automaticamente pelo programa sem necessidade de parsing manual.

O arquivo de saída, por sua vez, apresenta as principais informações finais do processo como a raiz encontrada, o erro associado, o número de iterações e o tempo de execução, possibilitando não apenas obter a solução, mas também comparar o desempenho entre diferentes métodos implementados

2.3 Dificuldades enfrentadas

Na implementação do método da bissecção, as dificuldades estiveram principalmente relacionadas a detalhes numéricos. Por exemplo, funções muito planas em torno da raiz, especialmente quando se adota uma tolerância muito pequena, aumentando o tempo de execução.

Outro ponto foi a escolha dos critérios de parada. Optou-se por usar tanto o valor absoluto da função no ponto médio quanto o tamanho do intervalo, de forma a evitar que o método continuasse iterando desnecessariamente em casos onde o valor da função demorasse a se aproximar de zero.

3. Método da Posição Falsa

3.1 Estratégia de implementação

O método da posição falsa foi implementado dentro da classe **Raízes**, junto com os outros métodos de busca de raízes, aproveitando a mesma estrutura de leitura do arquivo e organização do código. A lógica principal consiste em calcular, a cada iteração, um novo ponto aproximado para a raiz a partir da interpolação linear entre os extremos a e b.

meio = af(b) − bf(a) / f(b) − f(a)

O critério de parada é baseado tanto na proximidade do valor da função no ponto calculado em relação a zero quanto no número máximo de iterações definido pelo usuário.

Assim como em outros métodos, foram tomadas precauções contra erros numéricos, principalmente em situações onde f(a) e f(b) são muito próximos, o que poderia gerar divisões por zero. O código verifica continuamente a mudança de sinal da função para garantir que o intervalo continue válido, atualizando os extremos de acordo com a posição da raiz estimada.

3.2 Estrutura dos arquivos de entrada/saída

A entrada segue o mesmo padrão dos outros métodos, facilitando a reutilização dos dados. No arquivo **entrada.txt**, a primeira linha contém a função a ser analisada, seguida pelos valores iniciais aaa e bbb, e a tolerância desejada. Esse formato garante compatibilidade com os demais algoritmos da classe, permitindo alternar facilmente entre métodos diferentes sem necessidade de alterar o arquivo.

A saída do método é gravada em um arquivo chamado **saida_falsa_posicao.txt**, contendo as informações finais da execução: a raiz encontrada, o erro associado, o número de iterações realizadas e o tempo gasto. Esse formato simples possibilita comparar diretamente o desempenho da posição falsa com os demais métodos implementados.

3.3 Dificuldades enfrentadas

Uma das maiores dificuldades enfrentadas no método da posição falsa foi a questão da convergência. Em alguns testes, o método demorou muito para se aproximar da raiz, chegando a atingir o número máximo de iterações sem convergir satisfatoriamente. Isso ocorre porque a interpolação linear tende a privilegiar um dos extremos do intervalo, causando estagnação e dificultando o refinamento do valor aproximado.

Além disso, em situações em que a função assume valores muito pequenos, surgiram erros de arredondamento que prejudicaram a evolução do método.

4. Método Newton-Raphson

4.1 Estratégia de implementação

No método de Newton-Raphson, foi utilizada a biblioteca **SymPy** para calcular a derivada da função de forma simbólica e, em seguida, transformá-la em uma função numérica por meio do lambdify . Dessa forma, garantiu-se maior precisão na derivada, evitando erros que poderiam surgir caso fosse utilizada uma aproximação numérica.

Durante a implementação, foi considerada a possibilidade de a derivada ser igual a zero, o que inviabilizaria o cálculo devido à divisão por zero. Para lidar com esse problema, adicionou-se uma verificação que encerra a execução caso a derivada seja nula, evitando falhas ou instabilidades.

O critério de parada adotado baseou-se no valor absoluto da função no ponto atual: o método interrompe as iterações quando $|f(x)|$ é menor que a tolerância definida. Além disso, um limite máximo de iterações foi imposto, prevenindo a ocorrência de laços infinitos em situações de não convergência.

4.2. Estrutura dos arquivos de entrada/saída

A entrada para o método Newton-Raphson segue o mesmo formato utilizado pelos demais algoritmos implementados. O arquivo de entrada contém a função na primeira linha e, em seguida, os valores de a, b e a tolerância. Neste caso, os valores a e b são usados apenas para calcular um chute inicial, que é dado pela média aritmética entre eles. Assim, o formato da entrada permanece compatível entre os métodos, o que facilita a troca de estratégias de resolução sem a necessidade de alterar os dados de entrada.

A saída do programa é registrada em um arquivo separado, contendo as informações finais do processo: raiz encontrada, erro estimado, número de iterações realizadas e tempo de execução. Diferentemente do que ocorre em versões mais detalhadas do método, não são documentados os valores de cada iteração individual, mas apenas o resultado final, o que simplifica a análise do desempenho.

4.3. Dificuldades enfrentadas

Um dos maiores desafios da implementação foi lidar com situações em que a derivada se anula em algum ponto do processo iterativo. Nesses casos, o método falha, pois a divisão pelo valor da derivada resulta em indefinições matemáticas ou valores muito grandes, prejudicando a convergência. A verificação implementada para identificar a derivada nula foi essencial para evitar esse tipo de problema.

Outra dificuldade relevante foi a escolha do valor inicial para iniciar as iterações. Dependendo da função e do intervalo escolhido, o método pode divergir em vez de convergir para a raiz, o que torna a seleção de bons valores iniciais uma etapa importante. Ainda assim, a estrutura geral do método mostrou-se eficiente para funções bem comportadas, desde que atendidas as condições de convergência teóricas.

5. Método da Secante

5.1 Estratégia de implementação

O método da secante apresenta semelhanças com o de Newton-Raphson, mas com a vantagem de não necessitar do cálculo explícito da derivada. Em vez disso, a derivada é aproximada a partir de duas estimativas consecutivas da raiz, utilizando a diferença entre os valores da função nesses pontos. Isso simplifica a implementação, já que não há a necessidade de gerar uma função auxiliar para a derivada, tornando o método mais direto e independente da biblioteca simbólica.

Na implementação, foram tomadas precauções contra erros numéricos, principalmente em casos em que $f(b)-f(a)$ poderia ser zero, o que levaria a uma divisão indefinida. Para lidar com essa situação, foi adicionada uma verificação que interrompe o processo caso essa condição seja detectada. O critério de parada adotado foi o valor absoluto da função no ponto calculado: o método é encerrado quando $|f(x)|$ é menor que a tolerância fornecida.

5.2. Estrutura dos arquivos de entrada/saída

A entrada do método da secante segue o mesmo formato utilizado para os demais métodos implementados. O arquivo contém a função na primeira linha e, em seguida, os valores iniciais a, b e a tolerância. Neste caso, tanto a quanto b são necessários, pois o método exige duas aproximações iniciais para iniciar o processo iterativo.

A saída do programa é registrada em um arquivo próprio, denominado **saida_secante.txt**, contendo as informações finais: raiz encontrada, erro estimado, número de iterações e tempo de execução. Ao contrário de versões mais detalhadas, o código não gera uma tabela com todos os valores intermediários, mas apenas o resultado final, o que simplifica a interpretação e permite comparar diretamente a eficiência do método em relação aos demais.

5.3. Dificuldades enfrentadas

Uma das principais dificuldades do método da secante foi lidar com situações em que a diferença $f(b)-f(a)$ se aproximava de zero, o que causava instabilidade devido a divisões por valores muito pequenos. Para mitigar esse problema, foi necessário implementar verificações que detectassem esse cenário e interrompessem o cálculo, evitando resultados incorretos.

Outro desafio esteve na escolha das aproximações iniciais. Dependendo da função e dos valores fornecidos para a e b, o método pode apresentar dificuldades de convergência, oscilando entre valores ou divergindo em vez de

se aproximar da raiz. Ainda assim, quando bem inicializado, o método da secante mostrou-se bastante eficiente, alcançando a raiz em menos iterações que outros métodos, como a bissecção, e sem a necessidade de calcular derivadas como no Newton-Raphson.

6. Problemas Estabelecidos para Raízes das Funções

6.1 Problema 1 (Página 97 Q 3.3)

Questão: Um amplificador eletrônico com acoplamento RC e três estágios em cascata tem uma resposta a um degrau unitário de tensão dada pela expressão:

$g(T) = 1 - (1 + T + T^2/2) \cdot e^{(-T)}$,

onde $T = t / RC$ é uma unidade de tempo normalizada. O tempo de subida de um amplificador é definido como o tempo necessário para sua resposta ir de 10% a 90% do seu valor final. Como $g(\infty) = 1$, é necessário calcular os valores de T para os quais:

- $g(T) = 0,1$
- $g(T) = 0,9$

Entradas 1 e 2:

```
1 - (1 + x + x**2/2)* e**(-x) - 0.1
0
2
1e-8
```

Figura 1 - Entrada 1 do Problema 1 (3.3) para Métodos de Raízes da Função

```
1 - (1 + x + x**2/2)* e**(-x) - 0.9
4
6
1e-8
```

Figura 2 - Entrada 2 do Problema 1 (3.3) para Métodos de Raízes da Função

Saídas 1:

Bissecção

```
Raiz: 1.1020653247833252
Erro: 6.991845991066725e-10
Iterações: 23
Tempo: 0.000137
```

Figura 3 - Saída 1 do Método de Bissecção do Problema 1 (3.3)

Posição Falsa

```
Raiz: 1.1020652800650763
Erro: 9.720057647655267e-09
Iterações: 11
Tempo: 0.000066
```

Figura 4 - Saída 1 do Método da Posição Falsa para o Problema 1 (3.3)

Newton-Raphson

```
Raiz: 1.1020653282493214
Erro: 8.571920950828371e-12
Iterações: 4
Tempo: 0.000039
```

Figura 5 - Saída 1 do Método de Newton-Raphson para o Problema 1 (3.3)

Secante

```
Raiz: 1.1020653282481154
Erro: 2.431388423929093e-13
Iterações: 7
Tempo: 0.000056
```

Figura 6 - Saída 1 do Método da Secante para o Problema 1 (3.3)

Saídas 2:

Bissecção

```
Raiz: 5.322320461273193
Erro: 8.534340997790046e-09
Iterações: 22
Tempo: 0.000073
```

Figura 7 - Saída 2 do Método de Bissecção para o Problema 1 (3.3)

Posição Falsa

```
Raiz: 5.322320402913256
Erro: 4.4994439146739396e-09
Iterações: 15
Tempo: 0.000105
```

Figura 8 - Saída 2 do Método da Posição Falsa para o Problema 1 (3.3)

Newton-Raphson

```
Raiz: 5.32232033783421
Erro: 1.5922041601834636e-09
Iterações: 4
Tempo: 0.000046
```

Figura 9 - Saída 2 do Método de Newton-Raphson para o Problema 1 (3.3)

Secante

```
Raiz: 5.322320212875934
Erro: 8.639382681474572e-09
Iterações: 5
Tempo: 0.000052
```

Figura 10 - Saída 2 do Método da Secante para o Problema 1 (3.3)

6.2 Problema 2 (Página 100 Q 3.6)

A equação abaixo permite calcular o ângulo de inclinação (α), com relação à superfície da Terra, necessário para que um míssil atinja um determinado alvo:

$\tan(\theta / 2) = (\text{sen}(\alpha) * \cos(\alpha)) / [(g * R / v^2) - \cos^2(\alpha)]$

Onde:

● α: ângulo de inclinação (em radianos) com a superfície da Terra;

● g: aceleração da gravidade, aproximadamente 9,81 m/s²;

● R: raio da Terra, aproximadamente 6.371.000 m;

● v: velocidade de lançamento do míssil, em m/s;

● θ: ângulo (em graus) medido a partir do centro da Terra entre o ponto de lançamento e o ponto de impacto.

Considere os seguintes valores:

● θ = 80°;

● A razão $v^2 / (g * R) = 1,25$, o que implica em uma velocidade de

lançamento aproximada de 8.840 m/s.

Com base nesses dados, determine o valor do ângulo de inclinação α que satisfaz a equação dada.

Entrada:

Devido à presença de múltiplas raízes, a função torna-se mais difícil de ser totalmente analisada em um único intervalo. Inicialmente, foi estabelecido o intervalo de 0.5 a 1.5, de modo a conter uma das raízes conhecidas. Essa escolha permitiu que os métodos de bissecção, posição falsa, Newton-Raphson e secante encontrassem uma solução dentro do intervalo, sem risco de erro devido à seleção inadequada da faixa. Posteriormente, o intervalo foi ajustado para [0.8, 1.8], garantindo que a raiz desejada continuasse incluída e, ao mesmo tempo, diminuindo consideravelmente o número de iterações em alguns métodos, como o Newton-Raphson, aumentando a eficiência da convergência sem comprometer a exatidão do resultado.

```
(sin(x)*cos(x))/(0.8 - cos(x)**2) - tan(80*pi/180/2)
0.8
1.8
1e-8
```

Figura 11 - Entrada para o Problema 2 (3.6) para os Métodos de Raízes da Função

Saídas:

Bissecção

```
Raiz: 1.0237621396780014
Erro: 5.254590096370748e-10
Iterações: 26
Tempo: 0.000164
```

Figura 12 - Saída do Método de Bissecção para o Problema 2 (3.6)

Posição Falsa

```
Raiz: 1.0237621419101646
Erro: 4.552483057196355e-09
Iterações: 17
Tempo: 0.000126
```

Figura 13 - Saída do Método da Posição Falsa para o Problema 2 (3.6)

Newton-Raphson

```
Raiz: 1.023762139908983
Erro: 4.263843056406813e-09
Iterações: 5
Tempo: 0.000063
```

Figura 14 - Saída do Método de Newton-Raphson para o Problema 2 (3.6)

Secante

```
Raiz: 1.023762139910828
Erro: 4.197753256107717e-12
Iterações: 7
Tempo: 0.000098
```

Figura 15 - Saída do Método da Secante para o Problema 2 (3.6)

6.3 Problema 3 (Página 100 Q 3.8)

Questão: Uma loja de eletrodomésticos oferece dois planos de financiamento para um produto cujo preço à vista é de R$ 162,00:

● Plano A: entrada de R$ 22,00 + 9 prestações iguais de R$ 26,50

● Plano B: entrada de R$ 22,00 + 12 prestações iguais de R$ 21,50

Qual dos dois planos apresenta a menor taxa de juros, sendo portanto melhor para o consumidor?

Sabe-se que a equação que relaciona os juros mensais (J) com o prazo (P), o valor financiado (VF) e a prestação mensal (PM) é dada por:

(1 − (1 + J)^(-P)) / J = VF / PM

a) Fazendo as substituições x = 1 + J e k = VF / PM, verifique que a equação acima pode ser reescrita da seguinte forma:

f(x) = k·x^(P+1) − (k + 1)·x^P + 1 = 0

b) Escreva a equação do item (a) para cada um dos planos e encontre um intervalo que contenha a raiz positiva x != 1.

Entrada 1

```
5.283018868*x**10 - 6.283018868*x**9 + 1
1.1
1.9
1e-8
```

Figura 16 - Entrada 1 para o Problema 3 (3.8) para Métodos das Raízes da função

Entrada 2

```
6.511627907*x**13 - 7.511627907*x**12 + 1
1.1
1.9
1e-8
```

Figura 17 - Entrada 2 para o Problema 3 (3.8) para Métodos das Raízes da função

Saídas 1 e 2

Bissecção

```
Raiz: 1.1222483098506926
Erro: 1.6846442463247513e-08
Iterações: 27
Tempo: 0.000018
```

Figura 18 - Saída 1 do Método de Bissecção para o Problema 3 (3.8)

```
Raiz: 1.1093783676624298
Erro: 5.5968495615843494e-08
Iterações: 27
Tempo: 0.000021
```

Figura 19 - Saída 2 do Método de Bissecção para o Problema 3 (3.8)

Posição Falsa

```
Raiz: 1.1218946279159958
Erro: 0.0024281067542801793
Iterações: 1000
Tempo: 0.000407
```

Figura 20 - Saída 1 do Método da Posição Falsa para o Problema 3 (3.8)

```
Raiz: 1.1051064005346407
Erro: 0.04704940696614912
Iterações: 1000
Tempo: 0.000376
```

Figura 21 - Saída 2 do Método da Posição Falsa para o Problema 3 (3.8)

Newton-Raphson

```
Raiz: 1.12224830740827
Erro: 4.543920795185841e-12
Iterações: 10
Tempo: 0.000012
```

Figura 22 - Saída 1 do Método de Newton-Raphson para o Problema 3 (3.8)

```
Raiz: 1.1093783629238012
Erro: 1.4059864383852982e-10
Iterações: 11
Tempo: 0.000015
```

Figura 23 - Saída 2 do Método de Newton-Raphson para o Problema 3 (3.8)

Secante

```
Raiz: 1.1222483073625567
Erro: 3.1530333899354446e-10
Iterações: 8
Tempo: 0.000022
```

Figura 24 - Saída 1 do Método da Secante para o Problema 3 (3.8)

```
Raiz: 1.1093783629185046
Erro: 6.256328788367682e-11
Iterações: 7
Tempo: 0.000010
```

Figura 25 - Saída 2 do Método da Secante para o Problema 3 (3.8)

7. Comparativo

A análise comparativa dos métodos numéricos de Bissecção, Newton-Raphson, Falsa Posição e Secante, aplicada aos três problemas propostos, evidencia diferenças significativas em termos de desempenho, tempo de execução e robustez. Embora todos tenham como objetivo comum a determinação de raízes de equações não lineares, o comportamento observado varia conforme a natureza da função em estudo e os intervalos iniciais escolhidos.

No Problema 1, relativo ao cálculo do tempo de subida de um amplificador, foram utilizados os intervalos (0,2) para g(T) = 0,1 e (4,6) para g(T) = 0,9. Em ambos os casos, todos os métodos convergiram para a solução correta. O método da bissecção apresentou maior número de iterações, resultado esperado de sua convergência linear baseada em divisões sucessivas do intervalo. O método da falsa posição, por utilizar interpolação linear, alcançou a raiz em menos iterações que a bissecção, porém não superou a eficiência dos métodos abertos. O método de Newton-Raphson destacou-se pela rapidez, devido à sua convergência quadrática, obtendo a solução em poucas iterações. O método da secante apresentou desempenho semelhante, aproximando-se da eficiência do Newton-Raphson, ainda que sem necessidade da derivada explícita. Este problema ilustra de maneira clara a diferença entre métodos de intervalo, que oferecem robustez mas maior custo computacional, e métodos abertos, que, quando bem inicializados, alcançam convergência significativamente mais rápida.

No Problema 2, referente à determinação do ângulo de inclinação no lançamento de um míssil, o intervalo (0,8;1,8) foi utilizado. Novamente, todos os métodos foram capazes de convergir para a raiz. O Newton-Raphson manteve sua superioridade em termos de tempo e número de iterações, seguido de perto pelo método da secante. O método da bissecção, apesar de mais lento, garantiu a convergência com segurança. Já o método da falsa posição apresentou um ritmo de convergência inferior, consequência da tendência do processo de interpolação em privilegiar um dos extremos do intervalo, o que retarda o avanço em direção à raiz.

O Problema 3, relacionado à comparação de planos de financiamento, apresentou o maior contraste entre os métodos. Nesse caso, o método da falsa posição não conseguiu atingir a tolerância estipulada, mesmo após o número máximo de iterações, devido à estagnação provocada pela repetida escolha de pontos próximos ao mesmo extremo do intervalo. Esse comportamento é característica clássica do método, que pode apresentar lentidão extrema ou falhar em situações desse tipo. A bissecção, ainda que mais lenta, conseguiu convergir, reforçando sua robustez. O método de Newton-Raphson novamente

se mostrou o mais eficiente, obtendo a solução em poucas iterações, e o método da secante apresentou desempenho satisfatório, oferecendo uma alternativa prática quando a derivada não está disponível.

De forma geral, pode-se afirmar que as diferenças de tempo e eficiência observadas estão diretamente relacionadas às taxas de convergência de cada método. A bissecção apresenta convergência linear, a falsa posição também, mas pode ser ainda mais lenta devido a estagnações; a secante possui convergência superlinear; e o Newton-Raphson apresenta convergência quadrática, justificando seu desempenho superior quando as condições iniciais são adequadas e a derivada é bem definida. Dessa forma, os métodos de intervalo (bissecção e falsa posição) destacam-se pela segurança e garantia de convergência, enquanto os métodos abertos (Newton-Raphson e secante) apresentam maior eficiência computacional, sendo preferíveis quando há maior confiança nos dados iniciais e no comportamento da função.

8. Método da Eliminação de Gauss

8.1. Estratégia de implementação

A implementação do método da Eliminação de Gauss foi organizada dentro da classe **SistemasLineares**, que reúne diferentes técnicas para a resolução de sistemas lineares. Essa abordagem orientada a objetos favorece a reutilização do código e garante consistência na interface entre os métodos, já que todos compartilham a mesma estrutura de leitura dos dados e de escrita dos resultados em arquivos de saída.

No caso da Eliminação de Gauss, o procedimento inicia-se com a leitura da matriz **A** e do vetor **b** a partir de um arquivo de entrada. O código então realiza a transformação da matriz para a forma triangular superior utilizando duas iterações aninhadas. A primeira percorre cada linha da matriz e, a partir dela, a segunda faz a eliminação das linhas subsequentes. Para isso, é calculado um fator m = A[i,k] / A[k,k], que é usado para atualizar tanto a matriz de coeficientes quanto o vetor de termos independentes, por meio dos comandos:

A[i,k:] −= m · A[k,k:], b[i] −= m · b[k]

Esse processo garante que os elementos abaixo da diagonal principal se tornem nulos, resultando em uma matriz triangular superior.

A obtenção da solução final ocorre por meio da substituição regressiva, partindo da última equação do sistema. Para cada incógnita xi, o valor é calculado subtraindo-se da constante do sistema o somatório dos produtos dos coeficientes conhecidos com as variáveis já determinadas. Essa etapa está representada no trecho:

x[i] = b[i] − np.dot(A[i,i+1:], x[i+1:]) / A[i,i]

Com isso, as soluções são obtidas de forma sequencial até a primeira incógnita.

8.2. Estrutura dos Arquivos de Entrada e Saída

O arquivo de entrada utilizado (**entrada2.txt**) contém primeiramente a dimensão do sistema (número de equações), seguida das linhas representando os coeficientes da matriz **A** e os valores do vetor **b**. Cada linha apresenta os coeficientes de uma equação e, ao final, o termo independente correspondente.

Como saída, o método gera um arquivo (**saida_gauss.txt**) contendo a lista com a solução aproximada do sistema, além do tempo de execução. Esse

formato de saída foi escolhido para ser objetivo e de fácil interpretação, servindo também como registro para comparação entre diferentes métodos.

8.3. Dificuldades Enfrentadas

Um dos principais desafios da implementação foi garantir que o algoritmo não sofresse falhas ao se deparar com pivôs nulos durante a eliminação. Esse tipo de situação pode inviabilizar o escalonamento da matriz, levando a divisões por zero. Para contornar isso, foi necessário incluir verificações no código que interrompem a execução e retornam uma mensagem de erro caso o pivô seja nulo.

Outro ponto crítico foi lidar com erros de arredondamento inerentes às operações de ponto flutuante, especialmente em sistemas mal condicionados. Embora a utilização do **NumPy** proporcione maior estabilidade e eficiência no tratamento das matrizes, ainda foi preciso implementar arredondamento dos resultados na escrita da saída, garantindo clareza e consistência na apresentação final.

Por fim, foi importante avaliar o tempo de execução do método, mesmo em sistemas pequenos, de modo a possibilitar comparações futuras com métodos iterativos (como Jacobi e Gauss-Seidel).

9. Método Fatoração LU

9.1 Estratégia de Implementação

A implementação do método de Fatoração LU foi realizada dentro da classe **SistemasLineares**, que centraliza diferentes algoritmos de resolução de sistemas lineares. O procedimento inicia-se com a cópia da matriz de coeficientes **A** e do vetor **b**, de forma a preservar os valores originais. A matriz **L** é inicializada como identidade por meio do comando `np.eye(n)`, garantindo que sua estrutura seja mantida durante todo o processo.

Em seguida, é aplicado um laço duplo, onde para cada elemento abaixo da diagonal principal calcula-se:

L[j, i] = U[j, i] / U[i, i]

e a matriz **U** é atualizada com a operação:

U[j] −= L[j,i] · U[i]

Esse processo constrói simultaneamente as matrizes **L** (triangular inferior) e **U** (triangular superior) de forma eficiente, evitando cópias desnecessárias.

Com as matrizes definidas, a resolução do sistema é dividida em duas etapas. Primeiro, resolve-se o sistema triangular inferior Ly = b utilizando a relação:

y[i] = b[i] − np.dot(L[i,:i], y[:i])

Depois, aplica-se a substituição regressiva no sistema triangular superior Ux=y , com a fórmula:

x[i] = y[i] − np.dot(U[i,i+1:], x[i+1:]) / U[i,i]

Esse procedimento gera a solução final do sistema, que é então escrita em um arquivo de saída para posterior análise.

9.2. Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada2.txt**) segue o mesmo padrão utilizado na Eliminação de Gauss, apresentando inicialmente a dimensão do sistema, seguida de cada equação na forma dos coeficientes da matriz **A** e do termo independente de **b**.

A saída do método (**saida_lu.txt**) mantém a simplicidade e clareza, apresentando a solução calculada e o tempo de execução. Esse formato

permite comparação direta com os resultados obtidos por outros métodos, como a Eliminação de Gauss, Jacobi e Gauss-Seidel.

9.3. Dificuldades Enfrentadas

Na implementação da Fatoração LU, uma das dificuldades mais relevantes foi garantir a **estabilidade numérica** do algoritmo. Como a fatoração é sensível a erros de arredondamento, principalmente em matrizes com coeficientes de ordens de grandeza distintas, a escolha de pivôs não nulos é fundamental para evitar falhas. Para isso, o código inclui verificações que disparam exceções caso seja encontrado um pivô nulo, prevenindo divisões inválidas.

Outro desafio importante foi assegurar que as matrizes **L** e **U** fossem corretamente atualizadas em cada iteração, sem perda de informações durante a eliminação. A atualização simultânea das duas estruturas exigiu atenção ao controle dos índices, para evitar erros de propagação que poderiam comprometer a solução final.

Por fim, a gravação dos resultados em arquivo foi essencial não apenas como registro, mas também como ferramenta de validação e comparação entre métodos, permitindo verificar a consistência das soluções.

10.1. Problema 1 (Página 150 Q 4.1)

Considere o circuito a seguir com resistências e baterias tal como indicado; escolhemos arbitrariamente as correntes e os valores da malha:

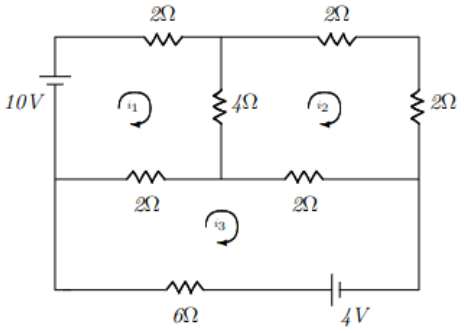

Figura 26 - circuito problema 1 (4.1)

Aplicando a Lei de Kirchoff que diz que a soma algébrica da diferenças de potencial em qualquer circuito fechado é zero, obtemos para as correntes i1, i2, i3, o seguinte sistema linear:

$$\begin{cases} 2\,i_1 + 4\,(i_1 - i_2) + 2\,(i_1 - i_3) - 10 = 0 \\ 2\,i_2 - 2\,i_2 + 2\,(i_2 - i_3) + 4\,(i_2 - i_1) = 0 \\ 6\,i_3 + 2\,(i_3 - i_1) + 2\,(i_3 - i_2) - 4 = 0 \end{cases}$$

Figura 27 - sistema linear do problema 1 (4.1)

Deseja-se determinar o valor de i = (i1, i2, i3)t que satisfaça o sistema acima.

Entrada

```
3
8 -4 -2 10
-4 6 -2 0
-2 -2 10 4
```

Figura 28 - Entrada para o problema 1 (4.1) para os métodos de sistemas lineares

Saídas

Gauss

```
Solução: [2.758621, 2.310345, 1.413793]
Tempo: 0.002066
```

Figura 29 - Saída do Método de Eliminação de Gauss para o problema 1 (4.1)

Lu

```
Solução: [2.758621, 2.310345, 1.413793]
Tempo: 0.000118
```

Figura 30 - Saída de Fatoração Lu para o problema 1 (4.1)

10.2. Problema 2 (Página 151 Q 4.3)

Um cachorro está perdido em um labirinto quadrado de corredores na Figura 31. Em cada intersecção escolhe uma direção ao acaso e segue até a intersecção seguinte onde escolhe novamente ao acaso nova direção e assim por diante. Qual a probabilidade do cachorro, estando na intersecção i, sair eventualmente pelo lado sul?

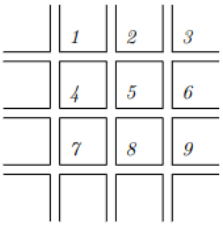

Figura 31 - labirinto do problema 2(4.3)

Esclarecimentos: Suponhamos que há exatamente as 9 interseções mostradas na figura. Seja P1 a probabilidade do cachorro, que está na intersecção 1, sair pelo lado sul. Seja P2, P3, . . . , P9 definidas de modo similar. Supondo que em cada intersecção a que chegue o cachorro, há tanta possibilidade que escolha uma direção como outra, e que, tendo chegado a uma saída, tenha terminado sua caminhada, a teoria das probabilidades oferece as seguintes equações para Pi:

$$\begin{cases} P_1 &= (0+0+P_2+P_4)/4 \\ P_2 &= (0+P_1+P_3+P_5)/4 \\ P_3 &= (0+P_2+0+P_6)/4 \\ P_4 &= (P_1+0+P_5+P_7)/4 \\ P_5 &= (P_2+P_4+P_6+P_8)/4 \\ P_6 &= (P_3+P_5+0+P_9)/4 \\ P_7 &= (P_4+0+P_8+1)/4 \\ P_8 &= (P_5+P_7+P_9+1)/4 \\ P_9 &= (P_6+P_8+0+1)/4 \end{cases}$$

Figura 32 - equações para Pi do problema 2 (4.3)

Entrada

```
9
4 -1 0 -1 0 0 0 0 0 0
-1 4 -1 0 -1 0 0 0 0 0
0 -1 4 0 0 -1 0 0 0 0
-1 0 0 4 -1 0 -1 0 0 0
0 -1 0 -1 4 -1 0 -1 0 0
0 0 -1 0 -1 4 0 0 -1 0
0 0 0 -1 0 0 4 -1 0 1
0 0 0 0 -1 0 -1 4 -1 1
0 0 0 0 0 -1 0 -1 4 1
```

Figura 33 - entrada para problema 2 (4.3)

Saídas

Gauss

```
Solução: [0.071429, 0.098214, 0.071429, 0.1875, 0.25, 0.1875, 0.428571, 0.526786, 0.428571]
Tempo: 0.000148
```

Figura 34 - saída do Método de Eliminação de Gauss para problema 2 (4.3)

Lu

```
Solução: [0.071429, 0.098214, 0.071429, 0.1875, 0.25, 0.1875, 0.428571, 0.526786, 0.428571]
Tempo: 0.000273
```

Figura 35 - saída do Método de Fatoração Lu para problema 2 (4.3)

10.3. Problema 3 (Página 153 Q 4.6)

Considere o circuito em escada, mostrado na Figura 36, composto de resistores concentrados e fontes de tensão ideais, uma independente e outra controlada.

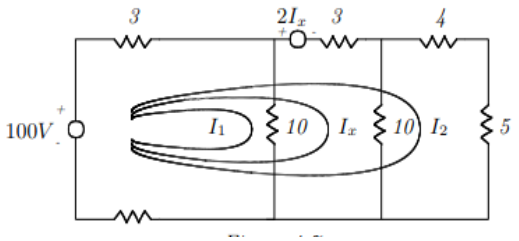

Figura 36 - Malha para resolução do problema 3 (4.6).

O sistema de equações que se obtém ao solucionar o circuito pelo método dos laços é o seguinte:

$$\begin{cases} 4\,(I_1 + I_2 + \ I_x) + 10\ I_1 = 100 \\ 4\,(I_1 + I_2 + I_x) + 3\,(I_2 + I_x) + 10\ I_x = 100 - 2\ I_x \\ 4\,(I_1 + I_2 + I_x) + 3\,(I_2 + I_x) + 9\ I_x = 100 - 2\ I_x \end{cases}$$

Figura 37 – Sistema linear do problema 3 (4.6).

Entrada

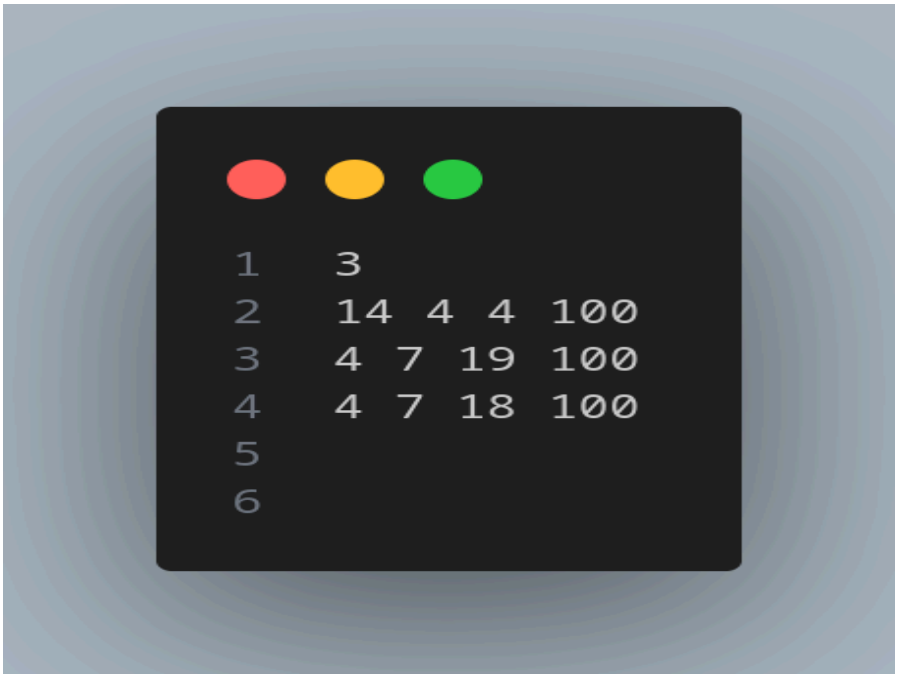

Figura 38 - entrada para o problema 3 (4.6).

Saídas

Gauss

```
Solução: [3.658537, 12.195122, -0.0]
Tempo: 0.000059
```

Figura 39 - saída do Método de Eliminação de Gauss para o problema 3 (4.6).

Lu

```
Solução: [3.658537, 12.195122, -0.0]
Tempo: 0.000072
```

Figura 40 - saída do Método Fatoração Lu para o problema 3 (4.6).

11. Comparativo

A análise comparativa entre os métodos de Eliminação de Gauss e Fatoração LU, aplicada aos problemas 4.1, 4.3 e 4.6, mostra que ambos são eficazes na obtenção da solução de sistemas lineares, mas apresentam diferenças importantes em relação ao desempenho computacional.

No Problema 4.1, observou-se um comportamento inesperado: a Eliminação de Gauss apresentou tempo de execução maior que a Fatoração LU. Essa inversão da tendência teórica deve-se ao pequeno porte da matriz, em que custos de manipulação de memória e operações adicionais influenciam mais que a complexidade algorítmica propriamente dita.

No Problema 4.3, os resultados alinharam-se à expectativa: a Eliminação de Gauss foi mais rápida que a Fatoração LU, uma vez que, para um único vetor independente, realiza diretamente a transformação para forma triangular e a resolução do sistema, sem necessidade da etapa adicional de decomposição.

O Problema 4.6 apresentou comportamento semelhante, com ambas as técnicas fornecendo soluções corretas, mas com a Fatoração LU novamente levando mais tempo devido ao custo adicional de construir as matrizes L e U.

De forma geral, conclui-se que a Eliminação de Gauss tende a ser mais eficiente quando o objetivo é resolver um único sistema linear. Já a Fatoração LU mostra-se vantajosa em cenários onde o mesmo sistema é resolvido diversas vezes para diferentes vetores bbb, pois a decomposição é realizada apenas uma vez, reduzindo o esforço nas soluções subsequentes.

12. Método de Jacobi

12.1 Estratégia de Implementação

A implementação do método iterativo de **Jacobi** foi realizada na classe **SistemasLineares**, a qual centraliza diferentes métodos para a solução de sistemas lineares. O procedimento inicia-se com a cópia da matriz de coeficientes **A** e do vetor de termos independentes **b**. Um vetor inicial **x** é criado com valores nulos, representando a primeira aproximação da solução.

O processo iterativo é controlado por um número máximo de iterações (max_iter) e por um critério de tolerância (tol). Em cada iteração, um novo vetor de aproximação (**x_novo**) é construído a partir da fórmula:

$$x_i^{(k+1)} = b_i - \sum_{j \neq i} A_{ij} \cdot x_j^{(k)} / A_{ii}$$

onde cada componente do vetor solução é atualizado com base apenas nos valores da iteração anterior. A convergência é verificada por meio da norma infinita da diferença entre o vetor novo e o anterior:

$$\| x^{(k+1)} - x^{(k)} \|_\infty$$

Se essa diferença for menor que a tolerância especificada, o processo é interrompido e a solução aproximada é retornada.

Esse método é simples e direto, porém não altera a matriz para torná-la diagonalmente dominante. Dessa forma, a convergência não é garantida para todos os sistemas, sendo dependente das propriedades da matriz de coeficientes. Os resultados obtidos ao final da execução são gravados em um arquivo de saída, permitindo comparação posterior com outros métodos.

12.2. Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada2.txt**) segue o mesmo padrão utilizado para os demais métodos, contendo inicialmente a dimensão do sistema, seguida das linhas com os coeficientes da matriz **A** e os valores correspondentes do vetor **b**. Isso garante a reutilização do mesmo arquivo em diferentes algoritmos de resolução.

A saída do método (**saida_jacobi.txt**) apresenta a solução final calculada, o número de iterações realizadas e o tempo de execução.

12.3. Dificuldades Enfrentadas

A principal dificuldade observada na implementação foi a **garantia de convergência**. O método de Jacobi depende fortemente das propriedades da matriz de coeficientes, e em casos onde ela não é **diagonalmente dominante** ou **simétrica definida positiva**, pode ocorrer divergência.

Outro desafio foi a escolha dos parâmetros de tolerância e número máximo de iterações. Valores muito restritivos podem interromper a execução antes da convergência, enquanto valores muito permissivos podem resultar em longos tempos de execução sem melhora real na precisão.

13. Método de Gauss - Seidel

13.1. Estratégia de Implementação

A implementação do método de **Gauss-Seidel** foi desenvolvida dentro da classe **SistemasLineares**, compartilhando a mesma estrutura de leitura dos dados e gravação dos resultados dos demais métodos. Assim como no Jacobi, o processo iterativo é iniciado com a criação de um vetor **x** inicializado com zeros. A diferença fundamental é que, no Gauss-Seidel, cada novo valor calculado para uma variável é utilizado imediatamente nas iterações subsequentes, acelerando o processo de convergência.

Em cada passo, o cálculo segue a fórmula:

$$x_i^{(k+1)} = b_i - \sum_{j<i} A_{ij} x_j^{(k+1)} - \sum_{j>i} A_{ij} x_j^{(k)}$$

onde os valores já atualizados na iteração corrente ($x_j^{(k+1)}$) são utilizados sempre que disponíveis. Esse procedimento aproveita as informações mais recentes, o que costuma resultar em convergência mais rápida quando comparado ao método de Jacobi.

O critério de parada é baseado na norma infinita da diferença entre o vetor atualizado e o vetor da iteração anterior. Caso essa diferença seja menor que a tolerância (tol), o processo é encerrado. Caso contrário, o método continua até atingir o número máximo de iterações (max_iter).

Ao final da execução, a solução aproximada e o número de iterações realizadas são gravados em arquivo, facilitando a validação e a comparação com outros métodos.

13.2. Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**entrada2.txt**) mantém o mesmo formato utilizado nos outros métodos: a primeira linha contém a dimensão do sistema e as linhas seguintes apresentam os coeficientes da matriz **A** junto com os termos do vetor **b**. Isso permite a reutilização do mesmo conjunto de dados para todos os algoritmos implementados.

O arquivo de saída (**saida_gauss_seidel.txt**) apresenta a solução aproximada do sistema, o número de iterações necessárias para a convergência e o tempo de execução.

13.3. Dificuldades Enfrentadas

A principal dificuldade enfrentada na implementação do Gauss-Seidel foi garantir a **atualização correta e imediata dos valores** do vetor solução. Como o método se diferencia do Jacobi ao utilizar as estimativas mais recentes dentro da mesma iteração, foi necessário ter cuidado especial com a manipulação dos índices para evitar inconsistências.

Outro desafio foi estabelecer um **critério de parada robusto**. Apesar de o Gauss-Seidel, em geral, convergir mais rapidamente que o Jacobi, sistemas mal condicionados ou matrizes que não são estritamente diagonais dominantes podem levar a oscilações ou divergências. Ajustar a tolerância e verificar o número adequado de iterações foram passos importantes para evitar paradas prematuras e garantir precisão nos resultados.

Por fim, a gravação da solução e das estatísticas de execução (tempo e número de iterações) mostrou-se essencial para a validação e para comparações posteriores com métodos diretos, como a Eliminação de Gauss e a Fatoração LU, bem como com o próprio Jacobi.

14. Problemas Estabelecidos de Sistemas Lineares: Métodos Iterativos

14.1. Problema 1 (Página 185 Q5.1)

$$\begin{pmatrix} 4 & -1 & 0 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 & -1 & 0 \\ 0 & -1 & 4 & 0 & 0 & -1 \\ -1 & 0 & 0 & 4 & -1 & 0 \\ 0 & -1 & 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 0 & -1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} 100 \\ 0 \\ 0 \\ 100 \\ 0 \\ 0 \end{pmatrix}.$$

Figura 41 - Sistema linear do problema 1 (5.1).

Entrada

```
6
4 -1 0 -1 0 0 100
-1 4 -1 0 -1 0 0
0 -1 4 0 0 -1 0
-1 0 0 4 -1 0 100
0 -1 0 -1 4 -1 0
0 0 -1 0 -1 4 0
```

Figura 42 - entrada para o problema 1 (5.1).

Saídas

Gauss-Seidel

```
Solução: [38.095238, 14.285714, 4.761905, 38.095238, 14.285714, 4.761905]
Iterações: 27
Tempo: 0.000530
```

Figura 43 - Saída do Método Gauss-Seidel para a resolução do problema 1 (5.1).

Jacobi

```
Solução: [38.095238, 14.285714, 4.761905, 38.095238, 14.285714, 4.761905]
Iterações: 51
Tempo: 0.000745
```

Figura 44 - Saída do Método de Jacobi para a resolução do problema 1 (5.1).

14.2. Problema 2 (Página 185 Q5.2)

Considere o circuito da figura a seguir, com resistências e baterias tal como indicado; escolhemos arbitrariamente as orientações das correntes.

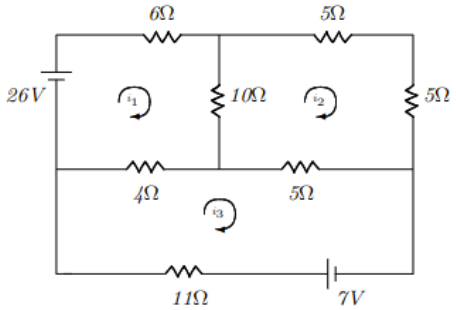

Figura 45 – Malha para resolução do problema 2 (5.2).

Aplicando a lei de Kirchoff, que diz que a soma algébrica das diferenças de potencial em qualquer circuito fechado é zero, achamos para as correntes i1,

i2, i3:

$$\begin{cases} 6i_1 + 10(i_1 - i_2) + 4(i_1 - i_3) - 26 = 0 \\ 5i_2 + 5i_2 + 5(i_2 - i_3) + 10(i_2 - i_1) = 0 \\ 11i_3 + 4(i_3 - i_1) + 5(i_3 - i_2) - 7 = 0 \end{cases}$$

Figura 46 – Sistema linear do problema 2 (5.2).

Entrada

```
3
20 -10 -4 26
-10 25 -5 0
-4 -5 20 7
```

Figura 47 – entrada do problema 2 (5.2).

Saídas

Gauss- Seidel

```
Solução: [2.0, 1.0, 1.0]
Iterações: 25
Tempo: 0.000541
```

Figura 48 – saída do Método de Gauss-Seidel para o problema 2 (5.2).

Jacobi

```
Solução: [2.0, 1.0, 1.0]
Iterações: 45
Tempo: 0.000488
```

Figura 49 – saída do Método de Jacobi para o problema 2 (5.2).

14.3. Problema 3 (Página 188 Q5.5)

Suponha que uma membrana com dimensões 80cm × 80cm tenha cada um dos seus lados mantidos a uma temperatura constante. Usando a teoria de equações diferenciais parciais pode-se formular uma equação que determina o valor da temperatura no interior dessa membrana. Essa equação diferencial pode ser simplificada colocando-se uma malha com 9 pontos sobre essa membrana e calculando-se a temperatura nos pontos da malha, como mostra a figura:

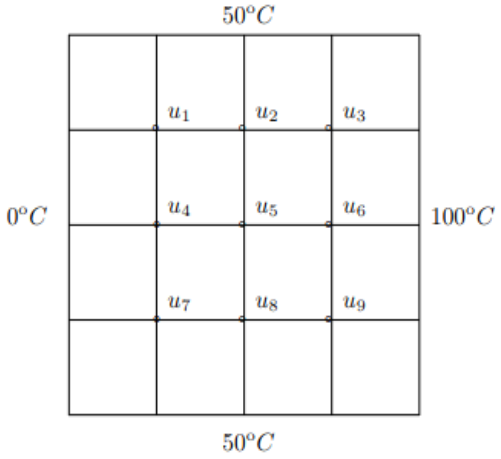

Figura 50 – Imagem para a resolução do problema 3 (5.5).

onde u1 , u2 , ..., u9 são os valores da temperatura em cada ponto da malha. O sistema de equações resultante é dado por:

$$\begin{pmatrix} -4 & 1 & & 1 & & & & & \\ 1 & -4 & 1 & & 1 & & & & \\ & 1 & -4 & & & 1 & & & \\ 1 & & & -4 & 1 & & 1 & & \\ & 1 & & 1 & -4 & 1 & & 1 & \\ & & 1 & & 1 & -4 & & & 1 \\ & & & & 1 & & -4 & 1 & \\ & & & & 1 & & 1 & -4 & 1 \\ & & & & & 1 & & 1 & -4 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_8 \\ u_9 \end{pmatrix} = \begin{pmatrix} -50 \\ -50 \\ -150 \\ 0 \\ 0 \\ -100 \\ -50 \\ -50 \\ -150 \end{pmatrix}.$$

Figura 51 – Sistemas de equação para o problema 3 (5.5).

Entrada

```
9
-4 1 0 1 0 0 0 0 0 -50
1 -4 1 0 1 0 0 0 0 -50
0 1 -4 0 0 1 0 0 0 -150
1 0 0 -4 1 0 1 0 0 0
0 1 0 1 -4 1 0 1 0 0
0 0 1 0 1 -4 0 0 1 -100
0 0 0 1 0 0 -4 1 0 -50
0 0 0 0 1 0 1 -4 1 -50
0 0 0 0 0 1 0 1 -4 -150
```

Figura 52 - entrada para o problema 3 (5.5)

Saída

Gauss-Seidel

```
Solução: [32.142857, 50.0, 67.857143, 28.571429, 50.0, 71.428571, 32.142857, 50.0, 67.857143]
Iterações: 40
Tempo: 0.001402
```

Figura 53 - saída do Método de Gauss-Seidel para o problema 3 (5.5)

Jacobi

```
Solução: [32.142857, 50.0, 67.857143, 28.571429, 50.0, 71.428571, 32.142857, 50.0, 67.857143]
Iterações: 76
Tempo: 0.003195
```

Figura 54 - saída do Método de Jacobi para o problema 3 (5.5)

15. Comparativo

A análise comparativa dos métodos iterativos de Jacobi e Gauss-Seidel, aplicada aos problemas 5.1, 5.2 e 5.3, evidencia diferenças significativas em termos de tempo de execução, número de iterações e eficiência de convergência. Ambos têm como objetivo comum a resolução de sistemas lineares por meio de aproximações sucessivas, mas a forma como utilizam as estimativas parciais influencia diretamente no desempenho.

No Problema 5.1, todos os métodos convergiram para a solução correta. O método de Jacobi apresentou um número de iterações consideravelmente maior, consequência de sua atualização baseada apenas nos valores da iteração anterior. O Gauss-Seidel, por sua vez, utilizou os valores mais recentes disponíveis dentro de cada iteração, resultando em convergência mais rápida e menor custo computacional.

O Problema 5.2 reforçou esse comportamento: enquanto Jacobi necessitou de várias dezenas de iterações para atender ao critério de tolerância, o Gauss-Seidel alcançou o mesmo resultado em quantidade significativamente menor, evidenciando sua maior eficiência prática.

No Problema 5.3, esse contraste foi ainda mais evidente. O Jacobi manteve seu padrão de convergência lenta, exigindo alto número de iterações, ao passo que o Gauss-Seidel convergiu rapidamente, confirmando sua vantagem na utilização de informações atualizadas ao longo do processo iterativo.

De forma geral, pode-se afirmar que, embora ambos garantam a convergência em sistemas diagonais dominantes, o método de Jacobi é mais simples e seguro, mas tende a ser mais lento. Já o método de Gauss-Seidel apresenta desempenho superior em termos de velocidade de convergência, sendo preferível quando se busca eficiência computacional sem perda de robustez.

16. Número de Condição de Matrizes

16.1 Estratégia de implementação

A implementação do código para calcular o número de condição da matriz foi estruturada de forma a priorizar clareza e eficiência. Para isso, utilizou-se a biblioteca **NumPy**, que fornece ferramentas otimizadas para operações matriciais e permite lidar com cálculos numéricos de forma mais estável e concisa. A escolha de aplicar a **fatoração LU pelo método de Doolittle** se mostrou estratégica, pois possibilita resolver sistemas lineares de maneira sistemática, além de facilitar a obtenção da matriz inversa, necessária para o cálculo do número de condição.

Outro ponto relevante foi a modularização das etapas em métodos distintos dentro da classe Matriz. Isso garantiu uma implementação mais organizada, separando claramente a leitura da entrada, a decomposição LU, a resolução de sistemas, o cálculo da inversa e, por fim, a avaliação do número de condição. Para aumentar a confiabilidade, foram inseridas verificações de erro simples, como a detecção de pivôs nulos, prevenindo divisões por zero durante o processo de decomposição.

Além disso, foi implementada a medição do tempo de execução do cálculo, fornecendo ao usuário uma noção clara do custo computacional envolvido. Por fim, os resultados — número de condição e matriz inversa — são armazenados em arquivo de saída, permitindo análise posterior sem a necessidade de reexecução imediata do programa.

16.2. Estrutura dos arquivos de entrada/saída

O arquivo de entrada **entrada3.txt** foi estruturado de forma simples, no qual cada linha representa uma linha da matriz, com os elementos separados por espaços. Essa escolha buscou manter a compatibilidade com a função de leitura em Python (split()), reduzindo a chance de erros de formatação. Esse formato torna a visualização intuitiva e facilita o uso por parte do usuário.

Já o arquivo de saída **saida_condicao.txt** contém, de forma organizada, o valor do número de condição calculado com cinco casas decimais, seguido da matriz inversa correspondente. Essa saída é apresentada em formato tabular, com cada linha representando uma linha da matriz inversa, o que assegura a legibilidade. O resultado gerado é consistente com a análise numérica esperada, já que a multiplicação de A · A^−1 ou AA^−1 retorna aproximadamente a matriz identidade.

16.3. Dificuldades enfrentadas

Durante a implementação, algumas dificuldades foram encontradas. A primeira esteve relacionada ao **cálculo da fatoração LU sem pivoteamento parcial**. Embora o algoritmo de Doolittle tenha sido implementado corretamente, a presença de pivôs nulos ou muito próximos de zero poderia comprometer a estabilidade numérica. Para contornar essa limitação, foi necessário adicionar verificações para impedir divisões inválidas.

Outra dificuldade relevante foi a **manipulação da matriz inversa**. Em versões iniciais do código, o resultado não era retornado adequadamente como uma matriz completa, o que prejudicava a escrita correta no arquivo de saída. Após ajustes, garantiu-se que cada coluna da inversa fosse calculada resolvendo sistemas auxiliares, o que assegurou a obtenção correta da matriz completa.

Por fim, foi necessário realizar diversos testes com diferentes matrizes, incluindo a fornecida em **entrada3.txt**, a fim de validar tanto a precisão numérica quanto a consistência do formato de entrada e saída. Esses testes confirmaram que a solução atende ao propósito proposto, gerando resultados corretos e de fácil interpretação.

17. Problemas Estabelecidos para Número de Condição de Matrizes

Condição das matrizes dos problemas de soluções lineares anteriores

17.1. 4.1

Entrada

```
8 -4 -2
-4 6 -2
-2 -2 10
```

Figura 55 - matriz de entrada para o código de matrizes mal condicionadas do problema 4.1

Saída

```
Número de condição: 5.98478
Matriz inversa:
0.24138 0.18966 0.08621
0.18966 0.32759 0.10345
0.08621 0.10345 0.13793
```

Figura 56 - condição da matriz do problema 4.1

17. 2. 4.3

Entrada

```
4 -1 0 -1 0 0 0 0 0
-1 4 -1 0 -1 0 0 0 0
0 -1 4 0 0 -1 0 0 0
-1 0 0 4 -1 0 -1 0 0
0 -1 0 -1 4 -1 0 -1 0
0 0 -1 0 -1 4 0 0 -1
0 0 0 -1 0 0 4 -1 0
0 0 0 0 -1 0 -1 4 -1
0 0 0 0 0 -1 0 -1 4
```

Figura 57 - matriz de entrada para o código de matrizes mal condicionadas do problema 4.3

Saída

```
Número de condição: 5.82843
Matriz inversa:
0.29911 0.09821 0.03125 0.09821 0.06250 0.02679 0.03125 0.02679 0.01339
0.09821 0.33036 0.09821 0.06250 0.12500 0.06250 0.02679 0.04464 0.02679
0.03125 0.09821 0.29911 0.02679 0.06250 0.09821 0.01339 0.02679 0.03125
0.09821 0.06250 0.02679 0.33036 0.12500 0.04464 0.09821 0.06250 0.02679
0.06250 0.12500 0.06250 0.12500 0.37500 0.12500 0.06250 0.12500 0.06250
0.02679 0.06250 0.09821 0.04464 0.12500 0.33036 0.02679 0.06250 0.09821
0.03125 0.02679 0.01339 0.09821 0.06250 0.02679 0.29911 0.09821 0.03125
0.02679 0.04464 0.02679 0.06250 0.12500 0.06250 0.09821 0.33036 0.09821
0.01339 0.02679 0.03125 0.02679 0.06250 0.09821 0.03125 0.09821 0.29911
```

Figura 58 - condição da matriz do problema 4.3

17.3. 4.6

Entrada

```
14 4 4
4 7 19
4 7 18
```

Figura 59 - matriz de entrada para o código de matrizes mal condicionadas do problema 4.6

Saída

```
Número de condição: 134.14125
Matriz inversa:
0.08537 0.53659 -0.58537
-0.04878 -2.87805 3.04878
-0.00000 1.00000 -1.00000
```

Figura 60 - condição da matriz do problema 4.6

17.4. 5.1

Entrada

```
4 -1 0 -1 0 0
-1 4 -1 0 -1 0
0 -1 4 0 0 -1
-1 0 0 4 -1 0
0 -1 0 -1 4 -1
0 0 -1 0 -1 4
```

Figura 61 - matriz de entrada para o código de matrizes mal condicionadas do problema 5.1

Saída

```
Número de condição: 4.04482
Matriz inversa:
0.29482 0.09317 0.02816 0.08613 0.04969 0.01946
0.09317 0.32298 0.09317 0.04969 0.10559 0.04969
0.02816 0.09317 0.29482 0.01946 0.04969 0.08613
0.08613 0.04969 0.01946 0.29482 0.09317 0.02816
0.04969 0.10559 0.04969 0.09317 0.32298 0.09317
0.01946 0.04969 0.08613 0.02816 0.09317 0.29482
```

Figura 62 - condição da matriz do problema 5.1

17.5. 5.2

Entrada

```
20 -10 -4
-10 25 -5
-4 -5 20
```

Figura 63 - matriz de entrada para o código de matrizes mal condicionadas do problema 5.2

Saída

```
Número de condição: 3.78371
Matriz inversa:
0.07090 0.03284 0.02239
0.03284 0.05731 0.02090
0.02239 0.02090 0.05970
```

Figura 66 - condição da matriz do problema 5.2

17.6. 5.5

Entrada

```
-4 1 0 1 0 0 0 0 0
1 -4 1 0 1 0 0 0 0
0 1 -4 0 0 1 0 0 0
1 0 0 -4 1 0 1 0 0
0 1 0 1 -4 1 0 1 0
0 0 1 0 1 -4 0 0 1
0 0 0 1 0 0 -4 1 0
0 0 0 0 1 0 1 -4 1
0 0 0 0 0 1 0 1 -4
```

Figura 65 - matriz de entrada para o código de matrizes mal condicionadas do problema 5.5

Saída

```
Número de condição: 5.82843
Matriz inversa:
-0.29911 -0.09821 -0.03125 -0.09821 -0.06250 -0.02679 -0.03125 -0.02679 -0.01339
-0.09821 -0.33036 -0.09821 -0.06250 -0.12500 -0.06250 -0.02679 -0.04464 -0.02679
-0.03125 -0.09821 -0.29911 -0.02679 -0.06250 -0.09821 -0.01339 -0.02679 -0.03125
-0.09821 -0.06250 -0.02679 -0.33036 -0.12500 -0.04464 -0.09821 -0.06250 -0.02679
-0.06250 -0.12500 -0.06250 -0.12500 -0.37500 -0.12500 -0.06250 -0.12500 -0.06250
-0.02679 -0.06250 -0.09821 -0.04464 -0.12500 -0.33036 -0.02679 -0.06250 -0.09821
-0.03125 -0.02679 -0.01339 -0.09821 -0.06250 -0.02679 -0.29911 -0.09821 -0.03125
-0.02679 -0.04464 -0.02679 -0.06250 -0.12500 -0.06250 -0.09821 -0.33036 -0.09821
-0.01339 -0.02679 -0.03125 -0.02679 -0.06250 -0.09821 -0.03125 -0.09821 -0.29911
```

Figura 66 - condição da matriz do problema 5.5

18. Considerações Finais

O desenvolvimento deste trabalho possibilitou uma análise ampla e detalhada de diferentes métodos numéricos aplicados à resolução de problemas matemáticos relevantes, incluindo o cálculo de raízes de funções e a solução de sistemas lineares. A comparação sistemática entre as técnicas destacou não apenas suas diferenças de desempenho em termos de tempo de execução e número de iterações, mas também a relevância de fatores como o condicionamento das matrizes e a natureza do problema estudado.

No caso dos métodos de busca de raízes, observou-se claramente a distinção entre os métodos de intervalo, como a bissecção e a falsa posição, e os métodos abertos, como Newton-Raphson e a secante. Enquanto os primeiros se mostraram mais robustos, ainda que mais lentos, os segundos se destacaram pela eficiência computacional, desde que bem inicializados. Essa análise reforçou a importância de considerar tanto a segurança de convergência quanto a rapidez desejada na escolha do método.

Na resolução de sistemas lineares, os experimentos com Jacobi e Gauss-Seidel evidenciaram a diferença no comportamento iterativo. O Jacobi apresentou convergência mais lenta, demandando um número elevado de iterações, enquanto o Gauss-Seidel aproveitou de forma mais eficaz as informações atualizadas, alcançando a solução em menos passos. Essa comparação mostrou, na prática, como métodos iterativos podem variar significativamente em desempenho mesmo diante do mesmo sistema.

Já entre os métodos diretos, a análise da Eliminação de Gauss e da Fatoração LU trouxe à tona a diferença entre eficiência imediata e reutilização de cálculos. A eliminação de Gauss se mostrou mais rápida para sistemas individuais, enquanto a fatoração LU destacou-se por sua aplicabilidade quando múltiplos vetores independentes precisam ser resolvidos para uma mesma matriz.

De forma integrada, os resultados ressaltam a importância de compreender o contexto e as características específicas de cada problema antes da escolha do método. Aspectos como número de iterações, tempo de execução, robustez e condicionamento da matriz desempenham papel crucial na estabilidade e confiabilidade dos resultados. Assim, a implementação prática dos algoritmos não apenas consolidou conceitos de álgebra linear e análise numérica, mas também evidenciou os desafios de equilíbrio entre precisão, eficiência e aplicabilidade.

Em termos mais amplos, este trabalho contribuiu para o desenvolvimento de uma visão crítica sobre a aplicação dos métodos numéricos, reforçando que não há uma técnica universalmente superior, mas sim soluções mais adequadas a cada cenário. A experiência adquirida constitui uma base sólida para estudos futuros, permitindo a expansão para métodos mais avançados,

além de aplicações em áreas como simulações numéricas, otimização e modelagem computacional. Conclui-se, portanto, que a análise numérica permanece como ferramenta indispensável para a prática científica e tecnológica, viabilizando soluções eficientes, precisas e confiáveis em problemas do mundo real.